Introduction to Programming

Lecture 14

VU
Virtual University

Code

```
calculateSalary ( int sal [ ] [ 2 ] , int lucky [ ] , int numEmps )
{
 for ( i = 0 ; i < numEmps ; i ++ )
 {
  // netSalary = grossSalary – tax
  if ( sal [ i ] [ 0 ] <= 5000 )
  {
   sal [ i ] [ 1 ] = sal [ i ] [ 0 ] ;
  }
```

Code

```
else
{
if ( sal [ i ] [ 0 ] <= 10000 )
   {
 sal [ i ] [ 1 ] = sal [ i ] [ 0 ] - 0.05*sal [ i ] [ 0 ] ;
   }
```

Code

```
else
{
if ( sal [ i ] [ 0 ] <= 20000 )
{
 sal [ I ] [ 1 ] = sal [ I ] [ 0 ] - 0.1 * sal [ I ] [ 0 ] ;
}
```

Code

```
else
{
    sal [ i ] [ 1 ] = sal [ i ] [ 0 ] - 0.15 * sal [ i ] [ 0 ] ;
    }
  }
 }
}
```

```
if ( sal [ i ] [ 0 ] >= 0 && sal [ i ] [ 0 ] <= 5000 )
{
 sal [ i ] [ 1 ] = sal [ i ] [ 0 ] ;
}
if ( sal [ i ] [ 0 ] > 5000 && sal [ i ] [ 0 ] < 10000 )
{
 sal [ i ] [ 1 ] = sal [ i ] [ 0 ] - 0.05 * sal [ i ] [ 0 ] ;
}
... ... ...
```

```
if ( grossSalary > sal [ i ] [ 0 ] && netSalary < sal [ i ] [ 1 ] )
```

This logic will fail

Code

```
void locateUnluckyIndividual ( int sal [ ] [ 2 ] , int lucky [ ] , int numEmps )
{
 int i , j ;
 int grossSalary , netSalary ;
  for ( i = 0 ; i < numEmp ; i ++ )
  {
  grossSalary = sal [ i ] [ 0 ]  ;
   netSalary    = sal [ i ] [ 1 ]  ;
 for ( j = 0 ; j < numEmp ; j ++ )
 {
  if ( grossSalary > sal [ j ] [ 0 ] &&  netSalary < sal [ j ] [ 1 ] )
  {
   lucky [ i ] = 1 ;
  }
 }
  }
}
```

Code

```
void displayOutput ( int sal [ ] [ 2 ] , int lucky [ ] , int numEmps )
 {
  for ( i = 0 ; i < numEmp ; i ++ )
 {
  if ( lucky [ i ] == 1 )
  {
   cout<< "Employee No." << i+1 << "
     is                                                                    unlucky,
     Gross Salary = " << sal [ i ] [ 0 ]    << " Net Salary = " << sal [ i ] [ 1 ] << "\n" ;
  }
 }
}
```

Pointers

Pointers

Location

x

60000

10

Address of x

Declaring Pointer to Integer

```
int *myptr ;
```

myptr is pointer to an integer

Declaring Pointers

```
double *x ;
char *c ;
```

Example

```
int *ptr ;
int x ;
x = 10 ;
ptr = &x ;
```

Dereferencing Operator *

*ptr is read as

"The value of what ever ptr points to"

```
z = *ptr * 2 ;
```

Initializing Pointers

```
ptr = &var ;
```

```
ptr = 0 ;
ptr = NULL ;
```

0 and NULL points to nothing

Example

```
main ( )
{
 int numEmp ;
 ....
 funct ( &numEmp ) ;
 ....
}

void funct ( int *numEmp )
{
     cin >> *numEmp ;
}
```

Declaring pointers

```
int *ptr1 , *ptr2 , *ptr3 ;
```

Declaring pointers

```
int *ptr , x ;
```

Declaring pointers

```
int *ptr , x , a [ 10 ] ;
```

Bubble Sort

5	1	1	1
1	5	3	2
3	3	5	3
6	6	6	4
9	2	2	5
2	9	4	6
4	4	8	8
8	8	9	9

Swapping

Swap

```
temp = x ;
x = y ;
y = temp ;
```

Example

```
main ( )
{
  int x = 10 , y = 20 , * yptr , * xptr ;
  yptr = &y ;
  xptr = &x ;
  swap ( yptr , xptr ) ;
}
```

Example

```
swap ( int *yptr , int *xptr )
{
    ... ... ...
}
```

const

```
int *const myptr = &x ;
```

myptr is a constant pointer to an integer

VU
Virtual University

const

```
const int x = 10 ;
```

const

```
const int *myptr = &x ;
```

myptr is a pointer to a constant integer

Array

```
int a[10];
```

Starting Address of Array **a**

1
2
3
4
5
6
7
8
9
10

VU
Virtual University